



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

Цель работы	3
Задание 1. Работа с массивами, статистика и обработка данных	3
Задание 2. Работа с таблицами в Pandas	11
Задание 3. Визуализация многомерных и категориальных данных	15
Задание 4. Скользящее окно на одномерных данных	19
Вывод.....	21

Цель работы

Цель практической работы №1 — научиться генерировать, анализировать и визуализировать данные с помощью Python, освоить базовые методы обработки числовых и категориальных признаков, применять статистические методы к данным.

Задание 1. Работа с массивами, статистика и обработка данных

1. Генерация данных

- сгенерируйте numpy-массив размером 100 с равномерным распределением от -100 до 50 (`numpy.random.uniform`);
- сгенерируйте массив из 100 случайных значений по нормальному распределению со средним 10 и $\sigma = 5$ (`numpy.random.normal`);
- сгенерируйте массив из 100 значений, распределённых по экспоненциальному закону с параметром $\lambda = 0.5$ (`numpy.random.exponential`).
- постройте гистограммы всех трёх массивов на одном графике (используйте `matplotlib` или `seaborn`), нормируйте плотность (`density=True`). Подпишите графики: укажите параметры, сравните формы распределений. Где «тяжёлый хвост»? Где симметрия?

Листинг 1 демонстрирует генерацию данных.

Листинг 1 — Код генерации данных

```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(21)
uniform_spread = np.random.uniform(-100, 50, 100)
```

Продолжение Листинга 1

```
normal_spread = np.random.normal(10, 5, 100)
expotential_spread = np.random.exponential(1 / 0.5, 100)

plt.figure(figsize=(12, 8))

plt.hist(uniform_spread, bins=20, density=True, label=f'Uniform spread (-100, 50)')
plt.hist(normal_spread, bins=20, density=True, label=f'Normal spread (10, 5)')
plt.hist(expotential_spread, bins=20, density=True, label=f'Expotential spread (0.5)')

plt.title('Spread comparison', fontsize=16, fontweight='bold')
plt.xlabel('Values')
plt.ylabel('Probability density')
plt.legend()

plt.grid()
plt.show()
```

Рисунок 1 демонстрирует график сравнения распределений.

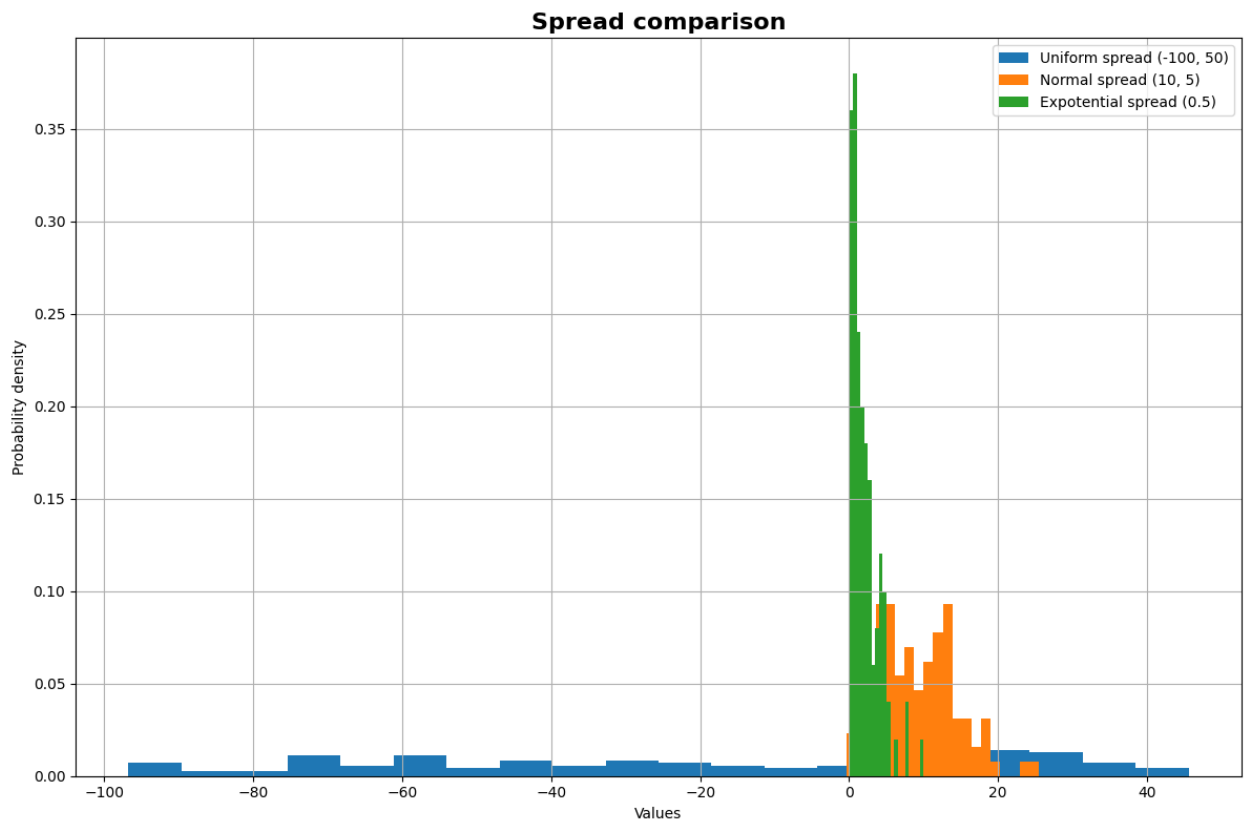


Рисунок 1 — Сравнение распределений. График

2. Алгебра, фильтрация и статистика

- реализуйте нормализацию z-score вручную, без использования `scikit-learn` и `stats.zscore`;
- реализуйте проверку на выбросы, например, выбросами будут

считаться элементы с $|z| > 2.5$;

- постройте гистограмму нормализованных значений и выделите выбросы другим цветом;
- отфильтруйте все элементы, находящиеся в пределах $\pm 1\sigma$, затем — в пределах $\pm 1,5\sigma$. Сравните количество таких элементов;
- вычислите вручную ковариацию, дисперсию и корреляцию Пирсона между двумя массивами;
- вычислите ковариацию, дисперсию и корреляцию Пирсона между двумя массивами с использованием NumPy (`numpy.cov`, `numpy.var`, `numpy.corrcoef`).

Код реализации z-score и построения масштабированных графиков представлен в Листинге 2.

Листинг 2 — Z-score и графики

```
def z_score(list):
    return (list - np.mean(list)) / np.std(list)

z_uniform = z_score(uniform_spread)
z_normal = z_score(normal_spread)
z_exp = z_score(exponential_spread)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
datasets = [z_uniform, z_normal, z_exp]
names = ['Uniform', 'Normal', 'Exponential']
colors = ['red', 'yellow', 'green']

for i, (data, name, color) in enumerate(zip(datasets, names, colors)):
    outliers = np.abs(data) > 2.5
    inliers = ~outliers

    axes[i].hist(data[inliers], bins=20, color=color, density=True)
    axes[i].hist(data[outliers], bins=20, color='black', density=True)
    axes[i].set_title(f'{name} spread')
    axes[i].set_xlabel('Z-score')
    axes[i].set_ylabel('Density')
    axes[i].legend()
    axes[i].grid()

plt.show()
```

Рисунок 2 демонстрирует масштабированные распределения.

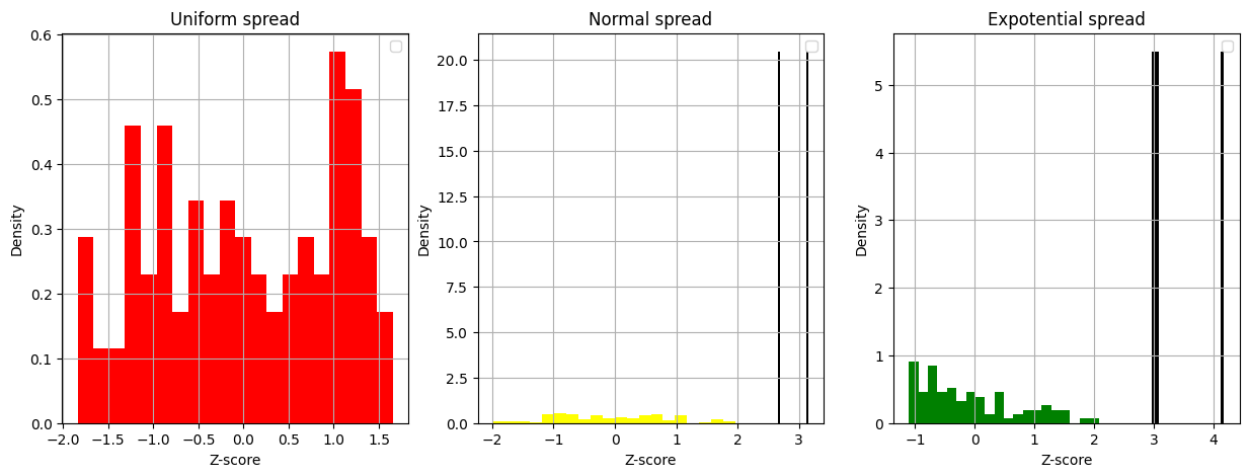


Рисунок 2 — Масштабированные распределения

Листинг 3 демонстрирует фильтрацию элементов в зависимости от стандартного отклонения, вычисление корреляции, ковариации, дисперсии.

Листинг 3 — Фильтрация элементов

```
print('The amount of elements in a range of diff sigmas:')
for i, (data, name) in enumerate(zip(datasets, names)):
    sigma_1 = np.sum(np.abs(data) <= 1)
    sigma_1_5 = np.sum(np.abs(data) <= 1.5)

    print(f'{name}:')
    print(f'Within the limits of 1: {sigma_1}')
    print(f'Within the limits of 1.5: {sigma_1_5}')

    print()

print('Manual calculation (uniform and normal spreads):')
x = uniform_spread
y = normal_spread

n = len(x)
mean_x = np.mean(x)
mean_y = np.mean(y)

var_x = np.sum((x - mean_x) ** 2) / (n - 1)
var_y = np.sum((y - mean_y) ** 2) / (n - 1)

cov = np.sum((x - mean_x) * (y - mean_y)) / (n - 1)
corr = cov / (np.sqrt(var_x) * np.sqrt(var_y))

print(f'Variance of a uniform spread: {var_x:.5f}')
print(f'Variance of a normal spread: {var_y:.5f}')
print(f'Covariation: {cov:.5f}')
print(f'Pearson Correlation: {corr:.5f}')
```

Получены результаты (см. Рисунок 3).

```

The amount of elements in a range of diff sigmas:
Uniform:
Within the limits of 1: 54
Within the limits of 1.5: 91

Normal:
Within the limits of 1: 67
Within the limits of 1.5: 86

Exponential:
Within the limits of 1: 69
Within the limits of 1.5: 93

Manual calculation (uniform and normal spreads):
Variance of a uniform spread: 1991.24887
Variance of a normal spread: 20.83600
Covariation: -30.51743
Pearson Correlation: -0.14982

```

Рисунок 3 — Вычисленные результаты

Далее идут те же характеристики, только вычисленные с помощью NumPy (см. Листинг 4).

Листинг 4 — Код для вычисления тех же характеристик с NumPy

```

print("Calculation using NumPy:")
var_x_np = np.var(x, ddof=1)
var_y_np = np.var(y, ddof=1)
cov_np = np.cov(x, y)[0, 1]
corr_np = np.corrcoef(x, y)[0, 1]

print(f'NumPy Variance of a uniform spread: {var_x_np:.5f}')
print(f'NumPy Variance of a normal spread: {var_y_np:.5f}')
print(f'NumPy Covariation: {cov_np:.5f}')
print(f'NumPy Pearson Correlation: {corr_np:.5f}\n')

print('Results comparison:')
print(f'Variance uniform: {np.isclose(var_x, var_x_np)}')
print(f'Variance normal: {np.isclose(var_y, var_y_np)}')
print(f'Covariation: {np.isclose(cov, cov_np)}')
print(f'Pearson Correlation: {np.isclose(corr, corr_np)}\n')

print('Outliers analysis:')
for i, (data, name) in enumerate(zip(datasets, names)):
    count = np.sum(np.abs(data) >= 2.5)
    percent = count / len(data) * 100
    print(f'{name}: {count}, {percent:.2f}%')

```

Результаты представлены на Рисунке 4.

```

Calculation using NumPy:
NumPy Variance of a uniform spread: 1991.24887
NumPy Variance of a normal spread: 20.83600
NumPy Covariation: -30.51743
NumPy Pearson Correlation: -0.14982

Results comparison:
Variance uniform: True
Variance normal: True
Covariation: True
Pearson Correlation: True

Outliers analysis:
Uniform: 0, 0.00%
Normal: 2, 2.00%
Exponential: 3, 3.00%

```

Рисунок 4 — Результат вычисленных характеристик

3. Срезы и условия

- выделите из равномерного массива все подпоследовательности длиной 3, в которых сумма > 30 : через цикл `for` и с помощью NumPy. Сравните производительность двух подходов;
- реализуйте вручную алгоритм вычисления экспоненциального сглаживания (ЕМА) с фактором затухания 0.2 для массива с нормальным распределением;
- реализуйте алгоритм вычисления экспоненциального сглаживания (ЕМА) с фактором затухания 0.2 через библиотеку Pandas;
- визуализируйте полученные ЕМА и сравните результаты.

Выделение подпоследовательностей длиной 3, в которых сумма больше 30, двумя способами представлено в Листинге 5.

Листинг 5 — Выделение подпоследовательностей двумя способами

```

def find_subs(arr):
    subs = []
    for i in range(len(arr) - 2):

```


Продолжение Листинга 5

```
        if np.sum(arr[i:i + 3]) > 30:
            subs.append(arr[i:i + 3])
    return subs

def find_subs_np(arr):
    windows = np.lib.stride_tricks.sliding_window_view(arr, 3)
    sums = np.sum(windows, axis=1)
    indices = np.where(sums > 30)[0]
    return windows[indices]

np.all(find_subs(uniform_spread) == find_subs_np(uniform_spread))
```

Результатом выполнения программы будет вывод `np.True`, означающий, что оба метода работают идентично.

Листинг 6 демонстрирует алгоритм вычисления ЕМА двумя способами.

Листинг 6 — Алгоритм вычисления ЕМА вручную и с помощью Pandas

```
def ema(arr, alpha = 0.2):
    values = np.zeros_like(arr)
    values[0] = arr[0]
    for i in range(1, len(arr)):
        values[i] = alpha * arr[i] + (1 - alpha) * values[i - 1]
    return values

def ema_pd(arr, alpha = 0.2):
    series = pd.Series(arr)
    return series.ewm(alpha=alpha, adjust=False).mean().values

print(f'Manual EMA: {ema(normal_spread)}')
print(f'Pandas EMA: {ema_pd(normal_spread)}')
```

Рисунки 5-6 демонстрируют графики визуализации полученных ЕМА.

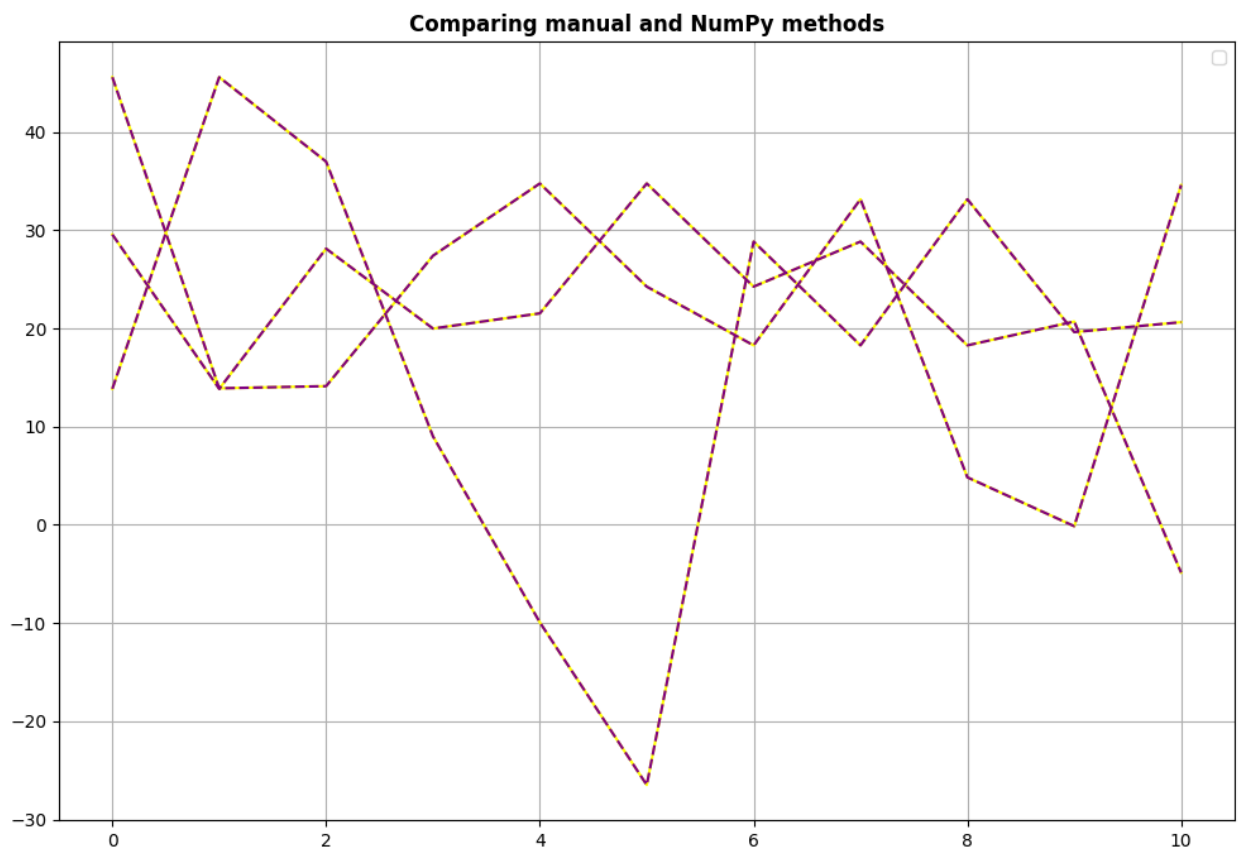


Рисунок 5 — Визуализация полученных ЕМА

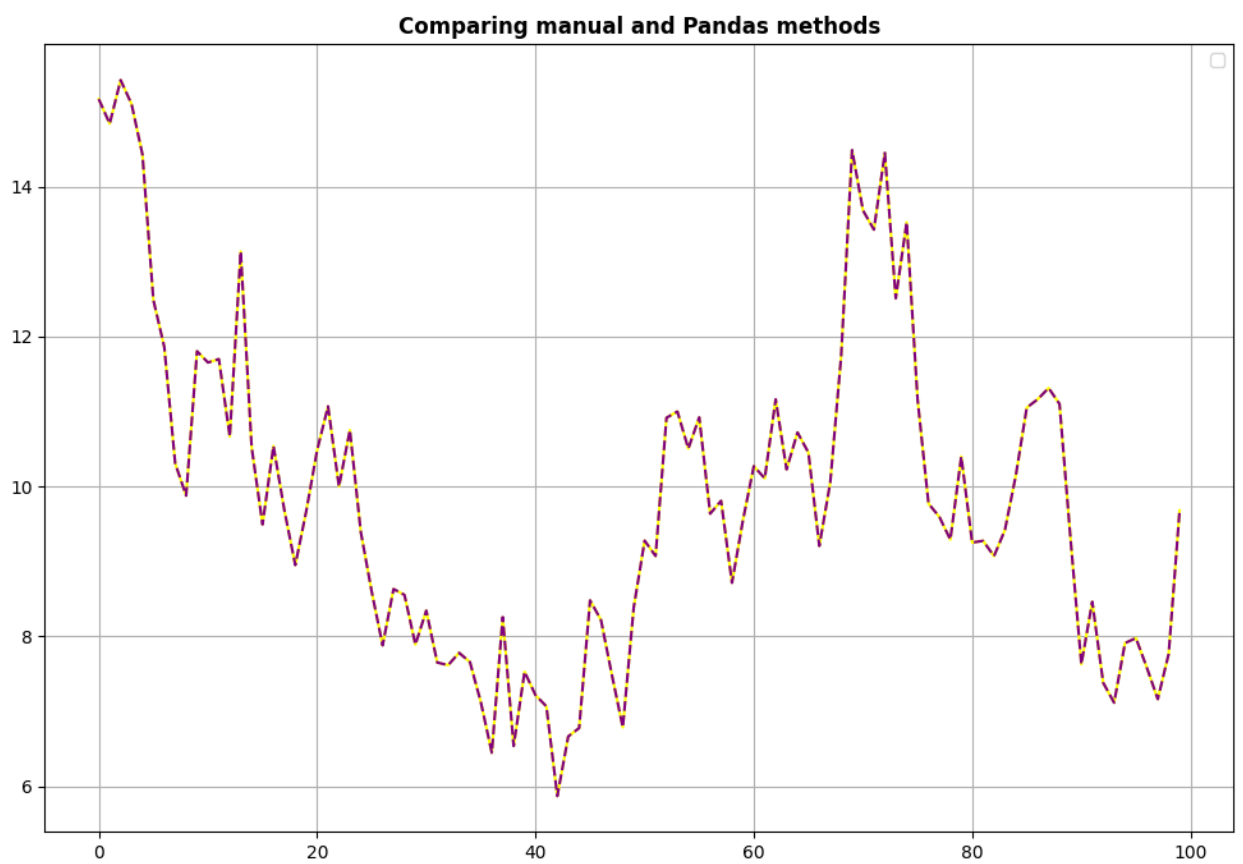


Рисунок 6 — Продолжение визуализации полученных ЕМА

4. Индексация и маски

- преобразуйте массив в матрицу 10×10 (reshape);
- установите в NaN (np.nan) те элементы, которые являются локальными минимумами по строкам.

Листинг 7 демонстрирует выполнение задания на индексацию.

Листинг 7 — Код задания на индексацию

```
reshaped = np.reshape(uniform_spread, (10, 10))
for i in range(reshaped.shape[0]):
    min_num = np.min(reshaped[i])
    for j in range(len(reshaped[i])):
        if reshaped[i][j] == min_num:
            reshaped[i][j] = np.nan
```

Задание 2. Работа с таблицами в Pandas

1. Создание DataFrame. Сгенерируйте таблицу на 500 строк с полями:

- user_id (уникальные ID);
- age (от 18 до 65);
- income (зависит от возраста: например, $\text{income} = \text{age} * \text{random.normal}(1.5, 0.2)$);
- subscription_date (случайные даты за последние 2 года);
- region (случайно: «Север», «Юг», «Центр», «Восток», «Запад»);
- is_active (булевый флаг активности пользователя).

Код программы генерации таблицы изображен в Листинге 8.

Листинг 8 — Генерация таблицы

```
from datetime import datetime, timedelta

rows = 500
ids = [_ for _ in range(1, rows + 1)]

ages = np.random.randint(18, 66, rows)

incomes = []
for i in range(rows):
    incomes.append(round(ages[i] * np.random.normal(1.5, 0.2), 2))

end_date = datetime.now()
```

Продолжение Листинга 8

```
start_date = end_date - timedelta(days=365*2)
dates = []
for i in range(rows):
    dates.append((start_date + timedelta(days=np.random.randint(0, 365 *
2))).date())

regions = ['Север', 'Юг', 'Центр', 'Восток', 'Запад']
regions_data = np.random.choice(regions, rows, p=[0.2, 0.2, 0.2, 0.2, 0.2])

is_active = np.random.choice([True, False], rows, p = [0.5, 0.5])

df = pd.DataFrame({'user_id': ids, 'age': ages, 'income': incomes,
'subscription_date': dates, 'region': regions_data, 'is_active': is_active})
df
```

Успешно сгенерированная таблица представлена на Рисунке 7.

	user_id	age	income	subscription_date	region	is_active
0	1	47	65.75	2024-10-23	Восток	False
1	2	23	37.49	2024-07-17	Центр	True
2	3	64	90.87	2024-01-04	Центр	False
3	4	51	105.07	2025-06-15	Центр	False
4	5	23	33.34	2024-09-16	Центр	False
...	...	...	...	...	...	...
495	496	40	52.89	2025-04-28	Восток	True
496	497	25	37.63	2025-04-09	Юг	False
497	498	27	36.86	2024-11-19	Запад	False
498	499	49	66.41	2024-10-03	Центр	False
499	500	22	36.31	2024-11-15	Юг	False

500 rows × 6 columns

Рисунок 7 — Успешно сгенерированная таблица

2. Работа с признаками

- вычислите `experience_months` — количество месяцев с момента подписки;
- создайте категориальный признак “`income_group`” по квантилям дохода;

- постройте новую метрику — `loyalty_score = income / (1 + experience_months)`.

Листинг 9 показывает вычисление `experience_moths`.

Листинг 9 — Вычисление `experience_months`

```
df['subscription_date'] = pd.to_datetime(df['subscription_date'])
df['experience_months'] = ((end_date - df['subscription_date']).dt.days /
30).round().astype(int)
```

Вычисление `income_group` изображено в Листинге 10.

Листинг 10 — Вычисление `income_group`

```
groups = ['Низкий', 'Средний', 'Высокий', 'Очень высокий']
quantiles = df['income'].quantile([0.25, 0.5, 0.75])
df['income_group'] = pd.qcut(df['income'], q=[0, 0.25, 0.5, 0.75, 1],
labels=groups)
df.head()
```

Вычисление `loyalty_score` (см. Листинг 11).

Листинг 11 — Вычисление `loyalty_score`

```
df['loyalty_score'] = (df['income'] / (1 + df['experience_months'])).round(2)
df.head()
```

3. Фильтрация и агрегации

- найдите средний доход активных пользователей по регионам;
- постройте сводную таблицу: средний `loyalty_score` по регионам и группам дохода;
- найдите 5% пользователей с наибольшим `loyalty_score`

Вычисление среднего дохода по регионам продемонстрировано в Листинге 12.

Листинг 12 — Вычисление среднего дохода по регионам

```
df_region = pd.DataFrame({'region': df['region'], 'income':
df['income']}).groupby('region').mean()
df_region
```

Вычисление среднего `loyalty_score` по регионам и группам дохода изображено в Листинге 13.

Листинг 13 — Вычисление среднего loyalty score по регионам и группа дохода

```
df_loyalty = pd.DataFrame({'region': df['region'], 'income_group':  
df['income_group'], 'loyalty_score': df['loyalty_score']}).groupby(['region',  
'income_group']).mean()  
df_loyalty
```

Листинг 14 показывает поиск 5% пользователей с наибольшим loyalty_score.

Листинг 14 — Поиск 5% пользователей с наибольшим loyalty score

```
df_highest = df[df['income_group'] == 'Очень высокий']  
df_highest = df_highest.sample(frac=0.05)  
df_highest
```

4. Обработка пропусков и ошибок

- случайно удалите 5% значений из income, замените их средним значением по региону;

Код программы к этому заданию представлен в Листинге 15.

Листинг 15 — Удаление 5% значений и замена их средним по региону

```
miss = int(len(df) * 0.05)  
miss_indices = np.random.choice(df.index, size=miss, replace=False)  
df.loc[miss_indices, 'income'] = np.nan  
  
income_region = df_region['income'].to_dict()  
df['income'] = df.apply(  
    lambda row: income_region[row['region']] if pd.isna(row['income']) else  
    row['income'], axis=1  
)  
df.head()
```

5. Контроль значений

- реализуйте функцию контроля значений: выбрасывайте строки, где income или age стали некорректными (например, отрицательными).

Код программы к этому заданию представлен в Листинге 16.

Листинг 16 — Функция контроля значений

```
def val_data(df):  
    income_val = (df['income'] > 0) & (df['income'].notna())  
    age_val = (df['age'] >= 18) & (df['age'] <= 65) & (df['age'].notna())  
    df_val = df[income_val & age_val].copy()  
    return df_val, len(df) - len(df_val)  
df, count = val_data(df)
```

Задание 3. Визуализация многомерных и категориальных данных

1. Обзор данных

- ознакомьтесь с описанием признаков;
- проверьте данные на наличие дубликатов и пропусков, устраните их при наличии;
- преобразуйте признаки cut, color, clarity в упорядоченные категории согласно логике качества (например, Fair < Good < Very Good < Premium < Ideal).

Исследование данных изображено в Листинге 17.

Листинг 17 — Исследование данных

```
diamonds = sns.load_dataset("diamonds")
diamonds.head()

diamonds.info()
```

2. Визуализация одномерных распределений

- постройте гистограмму распределения переменной price с логарифмическим масштабом;
- постройте boxplot для распределения price в зависимости от cut;
- постройте violinplot для price в зависимости от clarity и проанализируйте, где наблюдаются выбросы.

Визуализация одномерных распределений изображена на Рисунке 8.

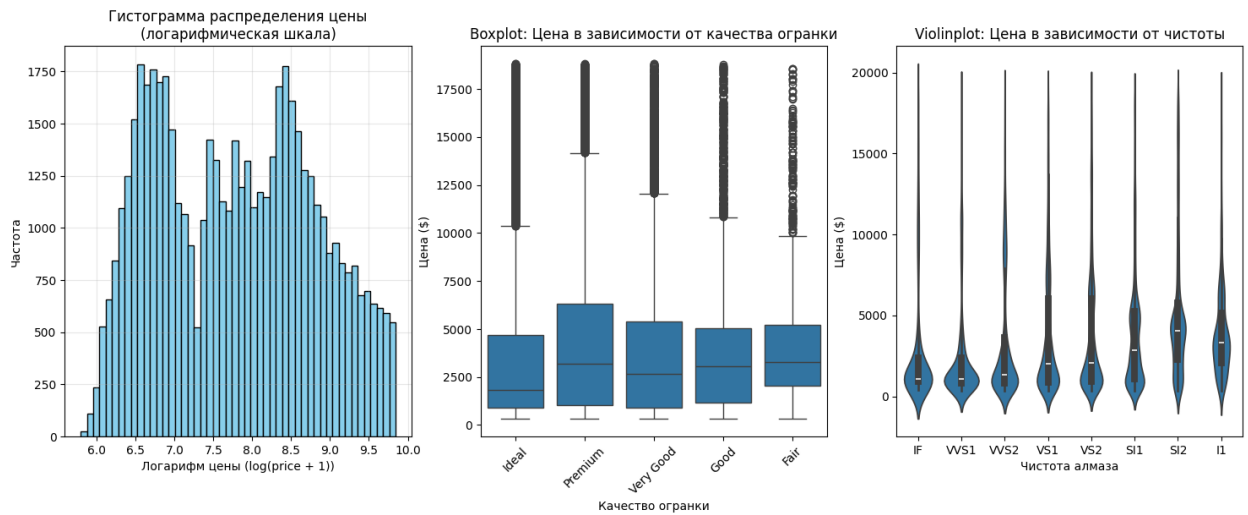


Рисунок 8 — Визуализация одномерных распределений

Анализ violinplot:

Наибольшее количество выбросов наблюдается в категориях с низкой чистотой (I1, SI2),

где есть много бриллиантов с высокой ценой, несмотря на низкое качество чистоты.

3. Анализ многомерных зависимостей

- постройте тепловую карту корреляции между числовыми признаками;
- постройте pairplot для признаков carat, price, depth, закодировав cut как цвет (hue);
- проанализируйте, какая комбинация признаков наиболее визуально предсказывает цену (price).

Рисунок 9 демонстрирует тепловую карту корреляции.

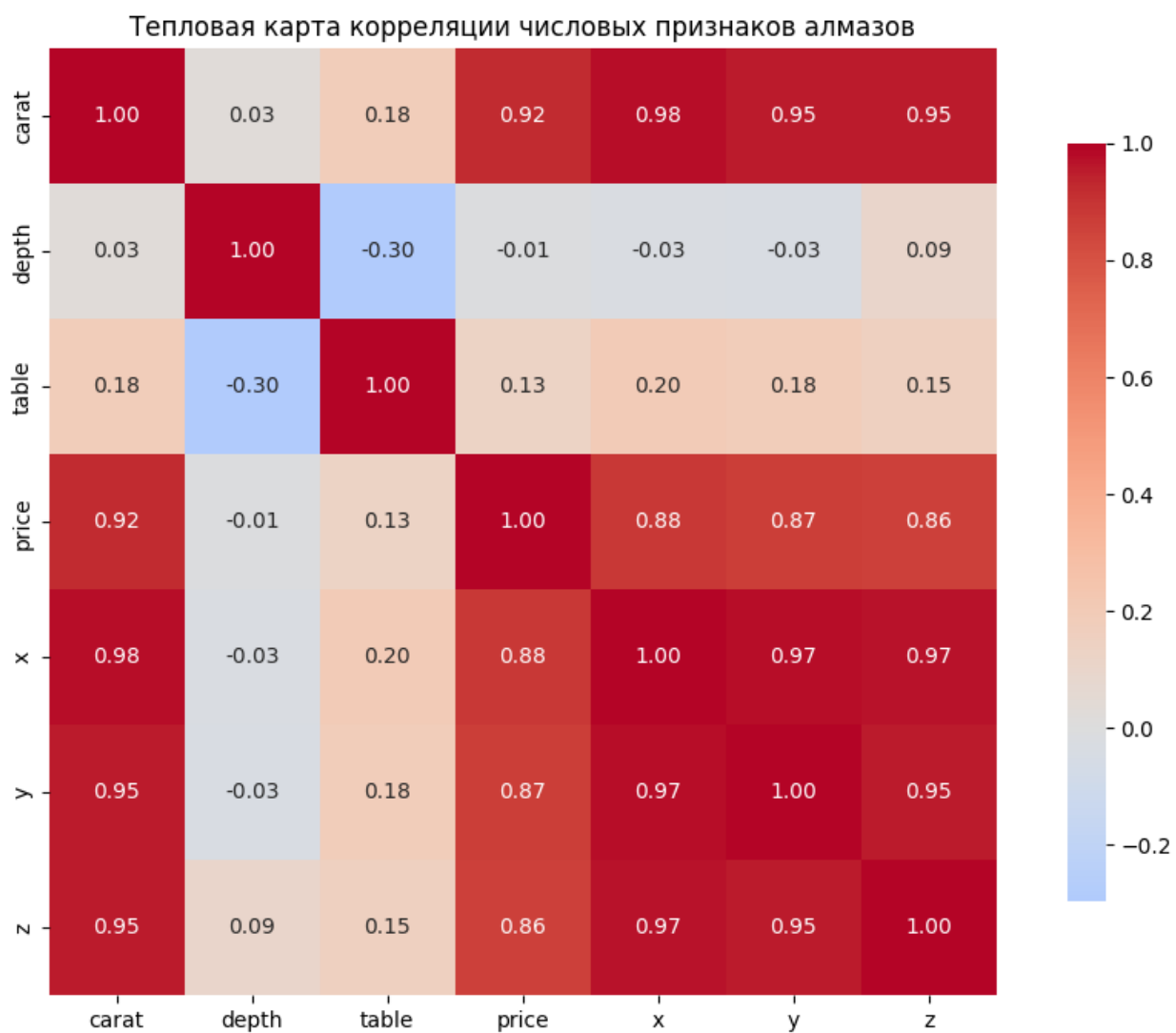


Рисунок 9 — Тепловая карта корреляции

На Рисунке 10 изображены pairplot графики.

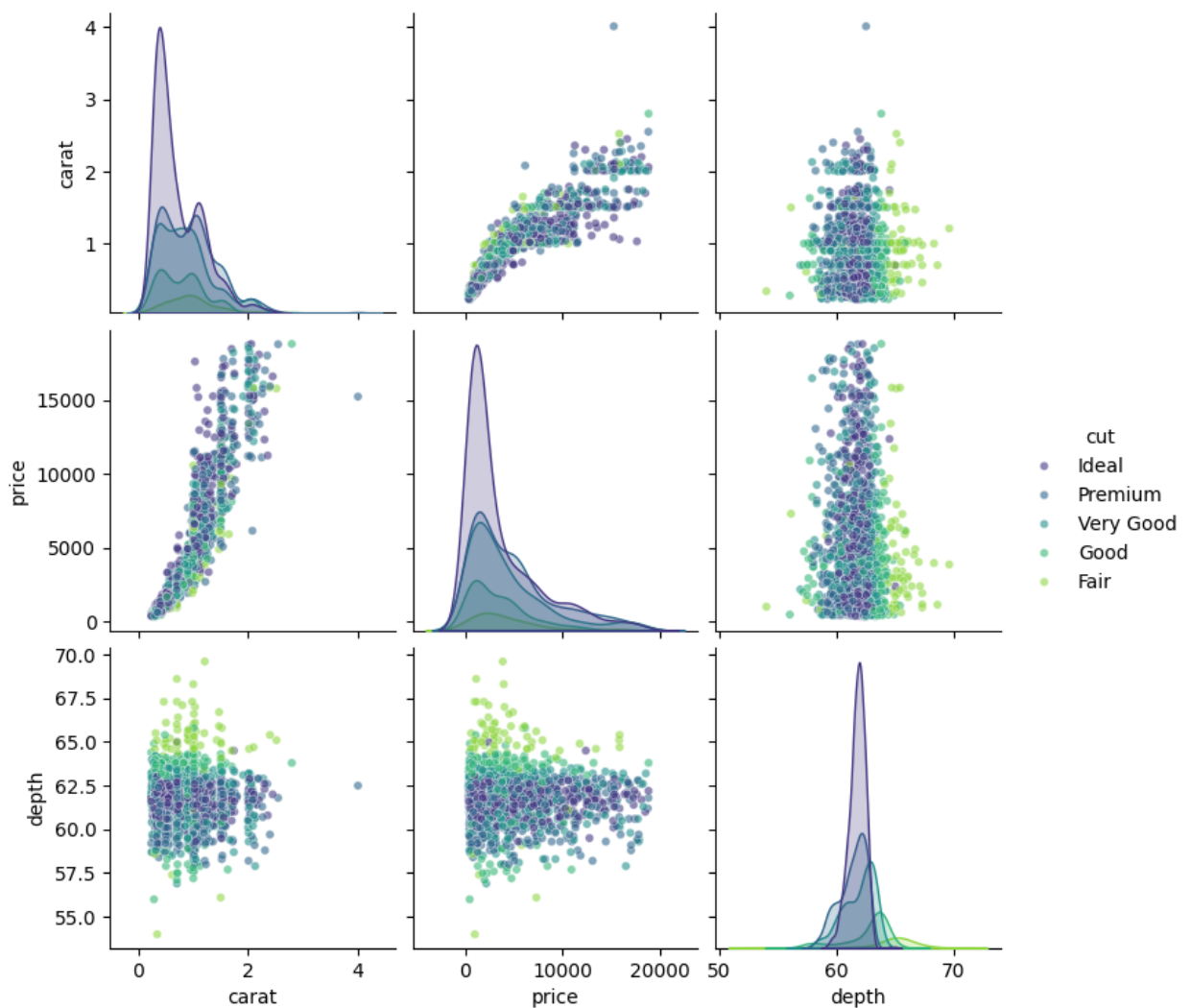


Рисунок 10 — Pairplot графики

Анализ pairplot:

Комбинация carat и price наиболее визуально предсказывает цену, так как наблюдается четкая положительная корреляция между этими признаками.

Бриллианты большего веса (carat) имеют тенденцию быть дороже.

4. Визуализация категориальных признаков

- постройте сводную таблицу, показывающую среднюю цену в зависимости от cut и color;
- На ее основе постройте тепловую карту и ответьте на следующие вопросы: Какой цвет и огранка в среднем наиболее дорогие? Есть ли закономерности? Если да, то какие?

Рисунок 11 показывает тепловую карту средней цены алмазов от качества огранки и цвета.

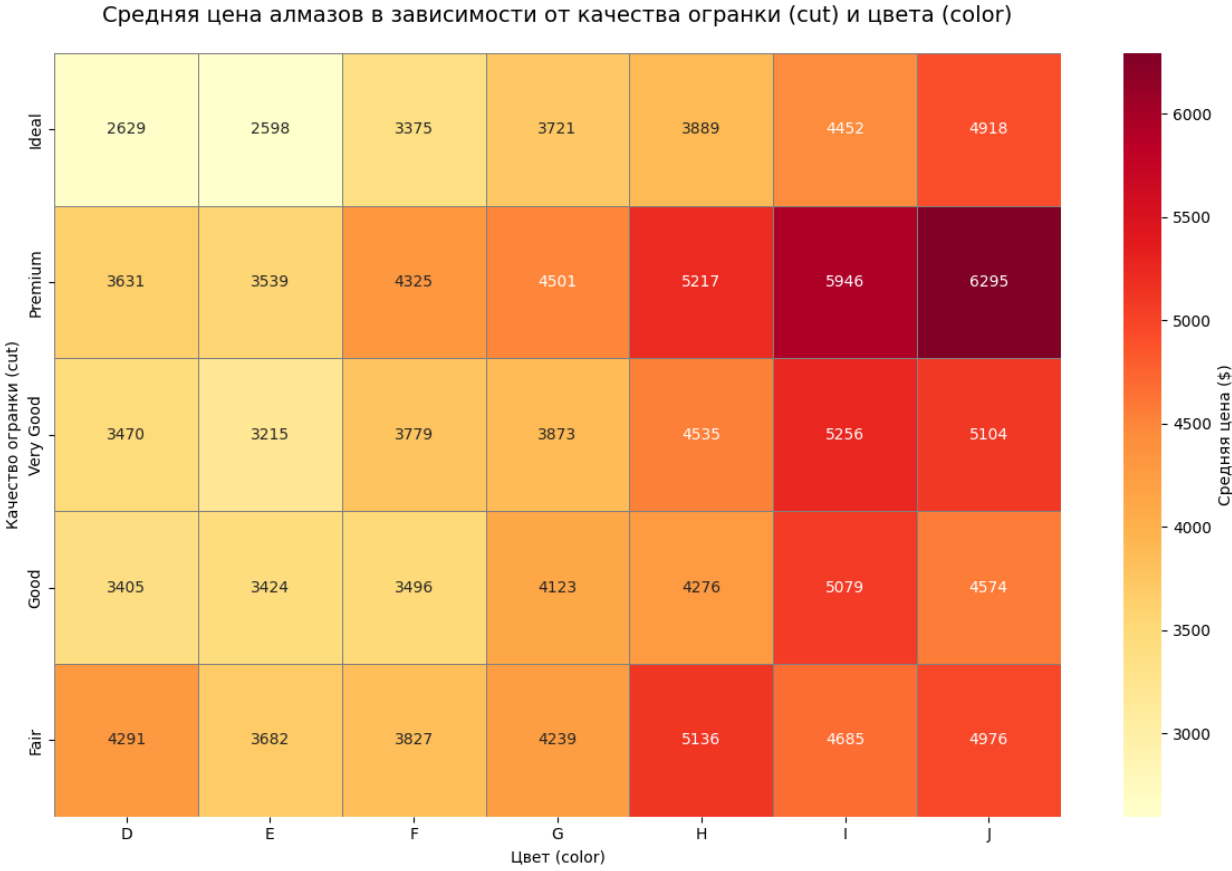


Рисунок 11 – Средняя цена бриллиантов по качеству огранки и цвету

Выводы:

Самый дорогой: Premium, J

Наибольшие выбросы цен наблюдаются в категориях с низкой чистотой.

Задание 4. Скользящее окно на одномерных данных

Необходимо реализовать функцию скользящего окна для заданного одномерного ряда данных, формирующую из него матрицу по следующему правилу (Рисунок 12).

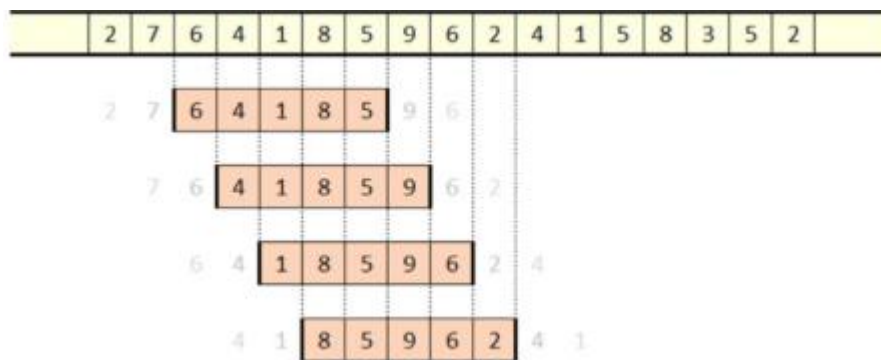


Рисунок 12 – Правило

Листинг 18 демонстрирует код программы скользящего окна и его тестирование.

Листинг 18 — Алгоритм скользящего окна

```
def sliding_window(x, w, step):
    slides = []
    for i in range(0, len(x) - w + 1, step):
        slides.append(x[i:i + w])
    return slides

window = 3
step_s = 1
x1 = np.array([8, 1, 4, 5, -2, 5, 9, 0])
A1 = np.array([[8, 1, 4],
[1, 4, 5],
[4, 5, -2],
[5, -2, 5],
[-2, 5, 9],
[5, 9, 0]])
print(np.array_equal(sliding_window(x1, w=window,
step=step_s), A1))
window = 2
step_s = 4
x2 = np.array([8, 3, 4, 1, -6, 5, 9, 2, 10, 11, -14, 0])
A2 = np.array([[8, 3],
[-6, 5],
[10, 11]])
print(np.array_equal(sliding_window(x2, w=window,
step=step_s), A2))
```

Оба теста выводят True, значит алгоритм реализован корректно.

Вывод

В ходе выполнения практической работы были получены навыки генерации, анализа и визуализации данных с помощью Python, освоены базовые методы обработки числовых и категориальных признаков, способность применять статистические методы к данным.